



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : *compare abstract class and interface*

Course title : Software Engineering and Project Management

Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report-1:

In Java, multiple inheritance means a class can get features from more than one parent. An abstract class does not support multiple inheritance. A class can extend only one abstract class. An ~~inherit~~ interface supports multiple inheritance in Java. A class can implement more than one interface at the same time. Abstract classes can have both abstract and normal methods.

Interfaces contain only abstract methods. We use an abstract class when classes are closely related. We use an interface when multiple inheritance is needed.

Therefore, interfaces are used to solve the multiple inheritance problem in Java.

Java code example:

```
interface A {  
    void show();  
}  
interface B {  
    void print();  
}
```



```
class Test implements A, B {  
    public void show() {  
        System.out.println("Interface A");  
    }  
    public void print() {  
        System.out.println("Interface B");  
    }  
}
```

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : *How does encapsulation ensure data integrity*

Course title : Software Engineering and Project Management

Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report- 2:

Encapsulation is an oop concept that hides data and protect them from direct access. It ensures data security by making variable private inside a class. Outside classes cannot change data directly. Data can be changed only using public methods. These methods check the values before saving them. This prevents wrong or harmful data from entering the system. Encapsulation also keeps data consistent and correct. If invalid data is given, the method rejects it. In bank system, this is very important for safety. Therefore, encapsulation ensures both data and security.

Java code:

```
class Account {  
    private String accountNumber;  
    private double balance;  
    public void setAccountNumber(String accNo)  
    { if (accNo != null && !accNo.isEmpty())
```

```
accountNumber = accno;
```

```
}
```

```
else {
```

```
System.out.println("Invalid account number");
```

```
}
```

```
}  
  
public void setInitialBalance(double amount) {
```

```
    if (amount  $\geq$  0) {
```

```
        balance = amount;
```

```
    }
```

```
    else {
```

```
        System.out.println("Balance cannot be  
negative");
```

```
    }
```

```
}
```

```
}
```




DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : *Developing multithreading-based proje*

Course title : Software Engineering and Project Management

Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report: 3

The program simulates a car parking management system using multithreading in java. Multiple cars request parking at the same time and their request are stored in shared synchronized queue. Parking agents run as separate threads and continuously take requests from the queue and park the cars. Synchronization ensures that no two agents park the same car at the same time.

Java code:

```
import java.util.*;
```

```
class parkingpool {
```

```
    Queue<String> q = new LinkedList<>();
```

```
    synchronized void add(String c) { q.add(c); notifyAll(); }
```

```
    synchronized String get() throws Exception {
```

```
        while (q.isEmpty()) wait();
```

```
        return q.poll();
```

```
    }
```

```
class ParkingAgent extends Thread {
```



```

ParkingPool P; String n;
ParkingAgent (ParkingPool P, String n) {this.P=P; this.n
=n;}
public void run() }
public class MainClass{
    public static void main(String[] a){
        ParkingPool P = new ParkingPool();
        new ParkingAgent (P, "Agent 1").start();
        new ParkingAgent (P, "Agent 2").start();
        System.out.println ("car ABC123 requested
        parking.");
        P.add ("ABC123");
        System.out.println ("car XYZ456 requested
        parking.");
        P.add ("XYZ456");
    }
}

```



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : *JDBC manages communication between java application.*
Course title : Software Engineering and Project Management
Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report #4;

JDBC stands for Java Database connectivity. It is used to connect a Java program with a relational database. JDBC works as a bridge between Java application and database. The Java program sends SQL queries using JDBC. JDBC driver receives the request and talks to the database. The database processes the query and sends the result back. JDBC driver receives (returns) the result to the Java program. JDBC allows data to be inserted, updated, deleted and read.

It also handles connection and error management. Thus, JDBC manages smooth communication between Java and database. To execute a SELECT query, JDBC follows

some fixed steps. First, a connection is created with the database. Then a statement prepared statement object is created. After that, a SELECT SQL query is written. The query is executed using executeQuery

Method. This method return a resultset.
try block is used to handle database operation
safety. catch block handles SQL on runtime
errors. finally block is used to close
connection and resources.

Java code:

```
import java.sql.*;
class SelectExample {
    public static void main(String[] args) {
        Connection con = null;
        try {
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root",
                "password");
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM Student");
            while(rs.next()) {
                System.out.println(rs.getInt(1) + " " + rs.getString(2));
            }
        } catch (Exception e) {
            System.out.println("Error occurred");
        }
    }
}
```



```
finally{  
try{  
    if(con!=null)  
        con.close();  
}  
catch (Exception e){  
    System.out.println("connection not closed");  
}  
}  
}
```

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY
SANTOSH, TANGAIL-1902



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : A servlet controllers manage the flow
Course title : between the model and the view.
: Software Engineering and Project Management
Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report-5:

In a java EE application, a servlet works as a controller. The controller manages the flow between model and view. The model contains business data and logic. The servlet gets data from the model. Then it sends this data to the view. Jsp is used as the view to show data to the user. The servlet forward the request to jsp using Request Dispatcher. Data is ~~on~~ sent using request attributes. Jsp reads the data and display it. Thus, the servlet controller controls the application flow.

Java code:

```
import java.io*  
import javax.servlet*;  
import javax.http.*;  
public class HelloServlet extends HttpServlet {  
    protected void doGet (HttpServletRequest req,  
                           HttpServletResponse res)  
        throws ServletException, IOException {  
        String name = "student";  
        req.setAttribute("msg", name);  
    }  
}
```

```
RequestDispatcher rd = req.getRequestDispatcher(
    "hello.jsp");
rd.forward(req, res);
}
```

JSP code for view:

```
<html>
```

```
<body>
```

```
<h2>Hello, $ {msg} </h2>
```

```
</body>
```

```
</html>
```




DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGIES

Lab Report

Name of Lab Report : *Example to insert a record into MySQL*

Course title : Software Engineering and Project Management

Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report 6:

Prepared statement is used to execute SQL queries safely in JDBC. It improves performance because the query is precompiled by the database. The same query can be used many times with different values.

Prepared statement is faster than statement for repeated queries. It also improves security by preventing SQL injection attacks. User input is treated as data, not as SQL and is less secure. Prepared statement uses placeholders (?) for values. These values are set using setter methods. So, prepared statement is better for the performance and security.

Java code:

```
import java.sql.*;
```

```
class insert {
```

```
    public static void main (String[] args) {
```

```
        try {
```

```
            Connection con = DriverManager.getConnection(
```

```
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
```



```
String sql = 'Insert into student (id, name) values (1)';  
PreparedStatement ps = con.prepareStatement(sql);  
ps.setInt(1, 1);  
ps.setString(2, "rahim");  
ps.executeUpdate();  
System.out.println("Record inserted");  
con.close();  
}  
catch (Exception) {  
    System.out.println("Error occurred");  
}  
}  
}
```

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : How JDBC retrieve data from MySQL

Course title : Software Engineering and Project Management

Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab report-7:

ResultSet is an object in JDBC that stores data returned from a database query. It is mainly used with select queries. ResultSet works like a table with rows and columns. The next() method moves the cursor to the next row, it returns true if data is available. The getString() method is used to read string type data. Data is read column by column from ResultSet. ResultSet helps Java programs fetch database records easily. Thus, it is very important for retrieving data in JDBC.

Java code:

```
import java.sql.*;
```

```
class ResultSetExample{
```

```
public static void main (String[] args){
```

```
try {
```

```

Connection con = DriverManager.getConnection
"jdbc:mysql://localhost:3306/testdb," "root",
"password");
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT id,
name from student");
while(rs.next()){
    int id = rs.getInt("id");
    String name = rs.getString("Name");
    System.out.println(id + " " + name);
}
con.close();
}
catch (Exception e){
    System.out.println("Error occurred");
}
}
}
}

```


MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Name of Lab Report : How to implemented REST

Course title : Software Engineering and Project Management

Course code : ICT-3108

Submitted By	Submitted To
Name: Md.Sahed ID: IT23036 3rd Year 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU	DR. ZIAUR RAHMAN Professor Department of ICT, MBSTU

Date of Submission :

Lab - 8

spring boot simplifies RESTful service development by providing auto-configuration, embedded servers and built-in JSON handling.

Java code:

```
import org.springframework.web.bind.annotation.*;  
import java.util.*;
```

```
@RestController
```

```
@RequestMapping("/api")
```

```
public class studentcontroller {
```

```
@GetMapping("/student")
```

```
public student getstudent() {
```

```
return new student(1, "Aren");
```

```
}
```

```
class student {
```

```
public int id;
```

```
public String name;
```

```
public student(int id, String name) {
```

```
this.id = id;
```

```
this.name = name;
```

```
}
```